

STUDY_PLAN.md — how to actually learn this project

Audience: a teammate who did not build this and needs to understand it deeply enough to defend it, extend it, or answer a hostile question without hesitating.

This is a curriculum, not a reference. Two other documents already exist and this one tells you *when* to read them:

Document	What it is	When you use it
<code>TEAMMATE_BRIEFING.md</code>	Pitch defence — headline numbers, judge Q&A	The night before presenting
<code>TECHNICAL_STUDY_GUIDE.md</code>	Reference — math, algorithms, code paths	Open beside the code, look things up
<code>STUDY_PLAN.md</code> (this)	Curriculum — ordered path with exercises	Start here, work through it

Total time: ~12 hours of focused work across 5 days for the core track. Role tracks add 3–5 hours each.

The single rule: do the *Break it* exercises. Reading about a causality guarantee teaches you nothing; deliberately breaking one and watching a test go red teaches you everything. Every session has one.

Table of contents

- Day 0 — Setup (45 min)
 - Day 1 — The problem and why the framing wins (2 h)
 - Day 2 — The bust label, the heart of the project (2.5 h)
 - Day 3 — Features, causality, and the poison test (2.5 h)
 - Day 4 — Model, calibration, baselines (2.5 h)
 - Day 5 — Explainability, refusal, serving (2 h)
 - Role tracks
 - Mastery check — 25 questions
 - Common misconceptions
 - Teach-back exercises
-

Day 0 — Setup (45 min)

Goal: a working environment where every command in this plan runs.

Prerequisites you genuinely need

You do **not** need meteorology. You need: - Python basics (functions, classes, pandas DataFrames) - What a classifier is, and what "probability of class 1" means - Comfort with a terminal

You do **not** need: deep learning, GIS experience, or NWP knowledge. Everything meteorological is explained where it appears.

Do this

```
git clone https://github.com/blazingarrows1525/forecast-bust-detection
cd forecast-bust-detection
pip install -r requirements.txt
```

Set `PYTHONPATH` for your shell (every command below assumes it):

```
export PYTHONPATH=src          # macOS/Linux
```

```
set PYTHONPATH=src
```

Verify the environment:

```
python -m pytest tests/ -q
```

Expected: `68 passed`. If you get import errors, `PYTHONPATH` is not set.

Get the dashboard data (63 MB, checksum-verified):

```
python scripts/fetch_release_artifacts.py
```

Start the server:

```
python -m uvicorn fbd.api.app:app --app-dir src --port 8912
```

Open `http://localhost:8912/` and `http://localhost:8912/command.html`. **Leave this running in its own terminal for the whole week.**

Day 0 checkpoint

You can answer: "*how many tests pass, and what are the two dashboard URLs?*" — 68, and `/` plus `/command.html`.

Day 1 — The problem and why the framing wins (2 h)

Goal: explain in your own words what the project predicts and why it isn't a weather forecaster.

Read (40 min)

- `README.md` top to bottom

- `TEAMMATE_BRIEFING.md` \$0, \$1, \$3
- `LOGIC.md` \$1 and \$2

Run (20 min)

```
python scripts/replay_demo.py
```

This narrates the June 2022 Assam & Meghalaya bust in the terminal. Watch what it prints — this is the story you will tell a judge.

Then open `http://localhost:8912/command.html` and click the tall red column over the northeast. Read the right panel.

Observe

Answer these from the screen, not from memory: - What probability did the model give Assam & Meghalaya at Day 4? - What did the ensemble-spread baseline give for the same cell? - What actually happened (forecast mm vs observed mm)?

The concept that takes longest to click

Most people first assume the system predicts **rainfall**. It does not. It predicts **whether someone else's rainfall forecast is wrong**. The input is a forecast; the output is a probability that *that forecast* fails.

Analogy that helps: it is not a weather forecaster, it is a **spell-checker for forecasts**. It never writes the sentence; it underlines the word that looks wrong.

Break it (20 min)

Open `http://localhost:8912/api/bulletin/ASSAM_MEGHALAYA?init_date=2022-06-14` in a browser. Find the JSON field `bust_probability`. Now find `forecast_rain_mm` and `observed_rain_mm`.

Question to sit with: the model produced `bust_probability` *without ever seeing* `observed_rain_mm`. Where in the pipeline does the observation enter, and why can it not enter earlier? (Answer on Day 2.)

Day 1 self-check

1. In one sentence, what does this system output?
2. Why is "we forecast rainfall better than IMD" a losing pitch in 2026?
3. What does "the ministry already did the inversion" mean?

ANSWERS

1. A calibrated probability, per subdivision per lead day, that the existing operational rainfall forecast for that cell will bust — plus a plain-language reason. 2. Because GraphCast/GenCast/Pangu already beat physics models on average scores; you would lose that comparison in one judge question. 3. The problem statement (SIH26079) itself asks for bust probability, not better forecasts — so answering it literally **is** the novel framing. No reframing risk.

Day 2 — The bust label, the heart of the project (2.5 h)

Goal: derive the three-condition bust definition from scratch and explain why each condition is necessary.

This is the most important day. Anyone can fit a classifier once labels exist. The labels are the research contribution.

Read (45 min)

- `src/fbd/labels/bust.py` — **the whole file**, docstring included. It is 156 lines.
- `TECHNICAL_STUDY_GUIDE.md` §2 (the derivation)

Run (30 min)

```
python -c "  
import sys; sys.path.insert(0,'src')  
from fbd.labels.bust import rain_category  
for mm in [0, 2.4, 2.5, 15.5, 15.6, 64.4, 64.5, 115.5, 204.5, 500]:  
    print(f'{mm:7.1f} mm -> category {rain_category([mm])[0]}')  
"
```

Now look at the real labelled data:

```
python -c "  
import sys; sys.path.insert(0,'src'); import pandas as pd  
d = pd.read_parquet('data/processed/dataset.parquet')  
print(d.groupby('split').bust.agg(['size', 'mean']).round(4))  
print()  
print('bust rate by lead day:')  
print(d.groupby('lead_day').bust.mean().round(4))  
"
```

Observe

The bust rate **rises monotonically with lead day** (about 2.6% at Day 1 to 5.8% at Day 10). That is physically correct — longer forecasts fail more — and it is *emergent*, not imposed. Nothing in the label definition mentions lead day.

Also note: **test-year bust rate (3.59%) is lower than train (4.07%)**. Sit with why that matters. If the P95 threshold were re-fitted per year, both would be pinned near 5% by construction.

The three conditions

Write them out by hand before reading further:

```
BUST = Magnitude ^ CategoryFlip ^ Significance
```

Magnitude: $|F - 0| \geq \max(10 \text{ mm}, P95_{\text{train}}(\text{subdivision}, \text{month}))$

CategoryFlip: $\text{Cat}(F) \neq \text{Cat}(0)$ using IMD's 6 official intensity bands

Significance: $\max(F, 0) \geq 15.6 \text{ mm}$ at least one side is "moderate" or above

Break it (45 min) — the important exercise

Open `tests/test_core.py` and find `test_bust_requires_all_three_conditions`. Read it.

Now deliberately weaken the definition. In `src/fbd/labels/bust.py`, find the line that computes the final label and remove the significance condition:

```
# find this
df["bust"] = (magnitude & category_flip & significant).astype("int8")
# change to
df["bust"] = (magnitude & category_flip).astype("int8")
```

Then:

```
python -m pytest tests/test_core.py -q
```

Watch it fail. Read the failure message. Then rebuild the dataset and see what happens to the bust rate:

```
python scripts/build_dataset.py
```

Compare the new bust rate to 4.07%. **Then undo your change** (`git checkout src/fbd/labels/bust.py`) and rebuild:

```
git checkout src/fbd/labels/bust.py && python scripts/build_dataset.py
```

Day 2 self-check

1. Why is a percentile-only definition circular?
2. Why does category-flip alone fail?
3. Where is P95 fitted, and what breaks if you fit it on all years?
4. Why do we require $\max(F, 0) \geq 15.6$ rather than ≥ 64.5 (heavy)?

ANSWERS

1. Taking the 95th percentile of `|error|` defines exactly 5% of rows as busts by construction. The "bust rate" then measures the definition, not the atmosphere. 2. It fires on trivial boundary straddles (15.5 vs 15.7 mm) and almost never fires in dry subdivisions whose whole record maxes out below the heavy threshold. 3. Training years only (2016–2020), in ``bust.error_thresholds()``. Fitting on all years pins the held-out bust rate at 5% and makes the evaluation meaningless. 4. Because area-averaging over a 19,000–222,000 km² subdivision dilutes extremes. A subdivision-mean of 64.5 mm/day is exceptionally rare; 15.6 mm/day already represents widespread significant rain and maps to a real advisory decision.

Day 3 — Features, causality, and the poison test (2.5 h)

Goal: explain the two causality axioms and prove they hold by breaking one.

Read (40 min)

- `src/fbd/features/forecast.py` — module docstring plus `lagged_ensemble()`
- `src/fbd/features/era5.py` — **the module docstring especially** (the causality rule)
- `TECHNICAL_STUDY_GUIDE.md` §3 and §10

The two axioms

Axiom A — no valid-time state. All ERA5 atmospheric features are sampled at initialisation time `t0`, never at the forecast's target day. The atmosphere on the verification day does not exist when the forecast is issued; using it would leak the answer.

Axiom B — lagged-ensemble causality. For lead `L`, the lagged ensemble uses only leads `≥ L`. A forecast at lead `L-1` verifying the same day was issued *later in time*.

Convince yourself of Axiom B with a diagram. Draw a timeline. Mark `t0`. Mark the valid day `V`. Now mark which initialisations produce a forecast for `V`, and which of those already exist at `t0`.

Run (30 min)

```
python -m pytest tests/test_core.py::test_lagged_ensemble_uses_only_leads_at_or_beyond_L -v
```

Then open that test and read it. It poisons leads 1–4 with the value 1000 and leaves leads 5–10 at 10, then asserts that leads 5–8 show *zero* spread — because their members are drawn only from `{L, L+1, L+2}`, all unpoisoned.

Break it (50 min) — the causality poison

In `src/fbd/features/forecast.py`, find `lagged_ensemble()` and locate:

```
cols = [lead_pos[l] for l in range(L, L + n_members) if l in lead_pos]
```

Change it to look **backwards** (the leak):

```
cols = [lead_pos[l] for l in range(L - n_members + 1, L + 1) if l in lead_pos]
```

Run:

```
python -m pytest tests/test_core.py -q
```

The poison test goes red. That is the guarantee working.

Now do the dangerous thing — see what the leak buys you:

```
git stash                                # save your break
git stash pop                            # reapply it
python scripts/build_dataset.py
python scripts/train_model.py
```

Look at the AUROC. **It will be higher.** This is the single most important lesson in the project: *a leak looks like success*. There is no error message, no crash — just a better number. The only thing standing between you and a fraudulent result is the test.

Undo:

```
git checkout src/fbd/features/forecast.py
python scripts/build_dataset.py && python scripts/train_model.py
```

Confirm AUROC returns to ~0.840.

Day 3 self-check

1. Why can't the model use the ERA5 analysis for the forecast's valid day?
2. For lead 5, which leads form the lagged ensemble, and why not lead 4?
3. What is the observable symptom of a causality leak?
4. Why does the lagged spread have **NaN** at Day 10?

ANSWERS

1. It doesn't exist yet when the forecast is issued. It would be a near-perfect predictor of whether the forecast busted — i.e. showing the model the answer. 2. Leads 5, 6, 7. Lead 4 verifying the same day was initialised one day *after* `t0`, so it is in the future relative to the prediction moment. 3. A *better* score with no error. Leaks never crash; they just inflate metrics. Only tests catch them. 4. The archive stops at 240 h, so leads 11 and 12 don't exist; Day 10 has only one member and spread is undefined. Disclosed in `DECISIONS.md` D-011, and it's why the headline is Day 3–7.

Day 4 — Model, calibration, baselines (2.5 h)

Goal: explain every hyperparameter, why isotonic calibration is mandatory, and why the baseline comparison is fair.

Read (40 min)

- `src/fbd/model/train.py` — the whole file
- `src/fbd/model/baselines.py`
- `TECHNICAL_STUDY_GUIDE.md` §12 and §16

The parameters that matter

Read them from the code, not from a slide (the docs have drifted; **code is truth**):

```
python -c "  
import sys, joblib; sys.path.insert(0, 'src')  
d = joblib.load('data/artifacts/bust_model.joblib')  
for k, v in d['params'].items(): print(f'{k:20s} {v}')  
print()  
print('n_features:', len(d['features']))  
print('calibrator:', type(d['calibrator']).__name__)  
"
```

The two lines a judge will ask about:

- `scale_pos_weight ≈ 25` — busts are ~4% of rows. Without this the model learns "always say no bust" and scores 96% accuracy while being useless.
- `IsotonicRegression` **fitted on the 2021 validation year** — because `scale_pos_weight` makes raw scores systematically over-confident. An uncalibrated bust probability is *worse than no probability*, because a forecaster would act on it.

Run (40 min)

```
python scripts/train_model.py
```

Read the whole output. Three tables print: overall, decision band, per-lead. Then:

```
python scripts/ablation.py
```

This shows what each feature group contributes. Note that **regime features add almost nothing to AUROC** — they are kept for explainability and independent validation, not for skill. Be ready to say that honestly.

The fairness protocol — why it matters

Every baseline gets the **same isotonic calibration** as the model, fitted on the same validation year. Read `scripts/train_model.py:Calibrated` to see it.

Without this, a class-weighted logistic regression emits ~0.5 probabilities and scores a Brier of 0.19 while having a perfectly respectable AUROC of 0.72. Beating that on Brier would prove nothing — it would be an artefact of the comparison.

Break it (30 min)

Remove the calibration and see what happens to reliability. In `src/fbd/model/train.py:predict_proba`, force the raw path:

```
def predict_proba(self, df):  
    return self.predict_raw(df)          # calibration bypassed
```

Then:

```
python scripts/train_model.py
```

Compare **ECE** (expected calibration error) before and after. It should jump from ~0.011 to something much larger. AUROC will barely move — **calibration changes probabilities, not ranking**. That distinction is worth internalising.

Undo: `git checkout src/fbd/model/train.py`

Day 4 self-check

1. What does `scale_pos_weight` do and why is it ~25 here?
2. Why isotonic rather than Platt scaling?
3. Why does calibration change Brier but barely change AUROC?
4. What is the honest margin over a real IFS ensemble, and why are there two numbers?

ANSWERS

1. It multiplies the gradient contribution of positive rows by ~25, effectively balancing a 4%/96% problem. Without it the model collapses to always predicting "no bust". 2. Platt fits a 2-parameter sigmoid and assumes the raw-score-to-probability relationship is sigmoid-shaped. `scale_pos_weight` warps scores in a way that violates that; isotonic makes no shape assumption. 3. AUROC is a **rank** statistic and isotonic calibration is **monotone**, so ordering is preserved. Brier measures squared probability error, which calibration directly targets. 4. +0.025 AUROC raw (0.832 vs 0.807) and +0.040 calibrated (0.832 vs 0.792). AUROC needs no calibration so the raw number is the honest AUROC comparison; the calibrated one is required for Brier/cost. Quote both — quoting only the friendlier invites a cherry-picking challenge (DECISIONS.md` D-014).

Day 5 — Explainability, refusal, serving (2 h)

Goal: explain how a SHAP number becomes an English sentence, and why refusing to answer is a feature.

Read (35 min)

- `src/fbd/explain/reasons.py` — the `TEMPLATES` dict and the family dedup logic
- `src/fbd/ood/detector.py` — the whole file, it is 99 lines
- `TECHNICAL_STUDY_GUIDE.md` §13, §14, §15

Run (35 min)

```
python scripts/stress_test.py
```

Look for the OOD table. The number to remember: **refused rows have a 23.4% bust rate vs 3.4% for accepted rows — about 7× higher.** That is what makes the refusal *earned* rather than decoration.

Then explore the reason engine live:

```
curl "http://localhost:8912/api/bulletin/ASSAM_MEGHALAYA?init_date=2022-06-14" | python -m json.tool
```

Find `dominant_factors` for lead 4. Three sentences, three *different concept families* — never three ways of saying "it's moist".

The two engineering tricks worth understanding

Concept-family dedup. SHAP often ranks `tcwv`, `moisture_flux_850`, and `mcz_q850_z` as the top three — all meaning "moist". The panel would say the same thing three times. Each feature maps to a family, and at most one factor per family is reported.

Direction-aware templates. A SHAP contribution can be positive for *either* direction of a feature. A single fixed wording produces sentences like "westerly flow is strong (−0.9 m/s, near normal)". Every template is a `(label, high_variant, low_variant)` triple.

Break it (30 min)

Find the `FAMILY` dict in `src/fbd/explain/reasons.py`. Map three moisture features to three *different* families (e.g. change `"tcwv": "moisture"` to `"tcwv": "moisture_a"`). Then:

```
python scripts/generate_bulletins.py --splits test
```

Query the same Assam row again and read `dominant_factors`. You should now see redundant moisture sentences. **That is what the dedup prevents.**

Undo: `git checkout src/fbd/explain/reasons.py` and regenerate.

Day 5 self-check

1. Why TreeSHAP and not KernelSHAP?
2. What makes an OOD refusal "earned"?
3. Why report only positive SHAP contributions in the reason panel?
4. What does `confidence_in_estimate` measure — and what does it *not* measure?

ANSWERS

1. TreeSHAP is exact and runs in $O(T \cdot L \cdot D^2)$ — milliseconds for our model. KernelSHAP is model-agnostic but $O(2^n)$ in features, requiring Monte-Carlo approximation with variance. For trees, exact beats approximate. 2. Refused rows bust $\sim 7\times$ more often than accepted rows (23.4% vs 3.4%). If refused rows busted at the same rate, the detector would be refusing at random and should be deleted. 3. The panel answers "why is confidence *low* here". Mixing in reassuring factors would bury the signal. 4. It measures how much the probability would move if the model had seen a slightly different training history (bootstrap-bagged refits). It does *not* measure whether the underlying weather is predictable — that's the bust probability itself.

Role tracks

After the 5-day core, pick the track matching what you'll own.

Track A — Data & Features (3 h)

Read `src/fbd/ingest/`, `src/fbd/regions/masks.py`, `src/fbd/features/era5.py`. Then `TECHNICAL_STUDY_GUIDE.md` §4, §5, §9, §11.

Key ideas to master: - Why exact polygon-cell overlap beats centroid masking (it silently drops Konkan & Goa — the highest-rainfall region) - The chunking economics: WeatherBench 2 zarr chunks span the globe, so subsetting to India saves memory but **not bandwidth** - Why the ERA5 store named `1959-2022` actually ends in 2021 (`DECISIONS.md` D-009)

Exercise: run `python -m fbd.regions.build` and confirm 641 districts partition into 36 subdivisions with no district assigned twice.

Track B — Modelling & Evaluation (4 h)

Read `src/fbd/evaluate/metrics.py` end to end. Then `TECHNICAL_STUDY_GUIDE.md` §17–§20.

Key ideas: - Why AUROC is class-balance invariant and why that matters at a 4% base rate - Murphy's Brier decomposition (reliability – resolution + uncertainty) - Equal-*count* reliability binning, not equal-width — with rare events, equal-width leaves upper bins nearly empty - The asymmetric cost: `C_miss = 10`, `C_false = 1`, and Richardson's economic value

Exercise: change the cost ratio in `src/fbd/config.py` from 10:1 to 3:1, re-run `train_model.py`, and observe how the optimal threshold and `value` shift.

Track C — Serving & Frontend (3 h)

Read `src/fbd/api/app.py`, `src/fbd/api/schema.py`, `web/index.html`, `web/command.html`. Then `TECHNICAL_STUDY_GUIDE.md` §21–§25.

Key ideas: - The Pydantic double-guard: request params *and* response model both validated - Two clock modes (`replay` vs `live`) and why live mode correctly reports STALE against a 2022 archive - The geometry-merge optimisation in the 3D view: 1,669 → 343 draw calls - Air-gap verification: every asset vendored, zero CDN

Exercise: add a new field to `BustPrediction` in `schema.py`, watch the API fail loudly until you populate it. That failure is the schema doing its job.

Track D — MLOps & Verification (3 h)

Read `src/fbd/mlops/drift.py`, `src/fbd/quality/`, `tests/` in full. Then `TECHNICAL_STUDY_GUIDE.md` §26, §27, and `DECISIONS.md` D-016, D-017.

Key ideas: - PSI (Population Stability Index) and what a PSI of 2.64 on `india_shear_z` means - Why the drift monitor saying RETRAIN on a year the model handled well is an *interesting* finding, not a bug - The verification ledger — what has and has not actually been run

Exercise: run `python scripts/monitor_drift.py` and explain each row of the output to another teammate.

Mastery check — 25 questions

Answer without looking. If you miss more than five, revisit the relevant day.

Problem framing 1. What does the system output, precisely? 2. Why is this not a weather forecasting model? 3. What did the ministry itself already do that removes our framing risk?

Labels 4. State all three bust conditions. 5. Why is percentile-only circular? 6. Where is P95 fitted? 7. Why is the test-year bust rate 3.59% and not 5%?

Data 8. Name the five datasets and their roles. 9. Why is IMD gauge data preferred over ERA5 for rainfall? 10. How many subdivisions are modelled, and why not 36? 11. What is the 3-hour offset problem?

Features 12. State both causality axioms. 13. For lead 5, which leads form the lagged ensemble? 14. What is the observable symptom of a leak? 15. Why does Day 10 have no lagged spread?

Model 16. What is `scale_pos_weight` and why ~25? 17. Why isotonic rather than Platt? 18. Why does calibration change Brier but not AUROC? 19. Name the four baselines. 20. What is the honest margin over real IFS ENS — both numbers?

Explainability & uncertainty 21. Why TreeSHAP over KernelSHAP? 22. What is concept-family dedup and why is it needed? 23. What makes the OOD refusal earned? 24. What does `confidence_in_estimate` actually measure?

Honesty 25. Name three things this project has *not* done.

ANSWER TO 25 (THE ONE PEOPLE FUMBLE)

No real AWS Bedrock invocation (account on Free Plan, GenAI layer tested only against a mock); Terraform never applied or even validated; no ECR/ECS deployment or public demo URL. All three are stated in `DECISIONS.md` D-017's verification ledger. Saying this confidently is a **strength** — it shows you distinguish what you ran from what you wrote.

Common misconceptions

Every one of these has been said by someone learning this project. Correct them early.

"It predicts rainfall." No. It predicts whether *someone else's* rainfall forecast will be wrong. The rainfall forecast is an input.

"Higher AUROC is always better." Not if it came from a leak. On Day 3 you will personally produce a *higher* AUROC by breaking causality. Score improvements must be explainable.

"The model uses observed rainfall as a feature." It does not, and cannot — observations don't exist at forecast time. `obs_rain_mm` appears in the dataset only to *construct the label*, never in the 52 features.

"We should have used deep learning." $\sim 10^5$ tabular rows, mandated explainability, and a 4% rare class. Trees win on all three. Being able to explain *why not* is worth more than using it.

"Regime features are the main contribution." Honestly, they add little AUROC (see `ablation.py`). They're kept for explainability and because they validate independently — depression days over Odisha show 25.1 mm/day vs 5.3. Say this honestly rather than overclaiming.

"The 96% accuracy is great." Never quote accuracy. At a 4% base rate, predicting "never bust" scores 96% and is worthless. This is why `LOGIC.md` §4.5 bans it.

"OOD refusal is us hedging." The opposite. Refused rows bust 7× more often. Refusing is the *correct* action on states unlike anything in training, and it's what a safety-critical tool should do.

Teach-back exercises

The fastest way to find out whether you actually understand something is to explain it to someone who will interrupt.

Exercise 1 — the 60-second pitch. Explain the project to a teammate who knows nothing, in 60 seconds, without saying "machine learning" or "AUROC". If they can't repeat back what it does, try again.

Exercise 2 — the label defence. Have a teammate play a hostile reviewer. They ask: "*isn't your bust definition arbitrary?*" You have three minutes. You must explain all three conditions and why removing any one breaks it.

Exercise 3 — the leak demo. Live-demo the Day 3 poison test to a teammate. Break causality, show AUROC rise, show the test go red, restore. This is the single most persuasive thing you can show a technically literate reviewer.

Exercise 4 — the honest answer. A teammate asks: "how do you know you didn't overfit the 2022 test year through your own iteration?" Practice the honest answer (see `TECHNICAL_STUDY_GUIDE.md` §32). Do not bluff. The honest answer earns more respect than a confident false one.

Exercise 5 — the live demo, cold. Run the full dashboard demo — 3D command centre, click the red column, read reasons, hit Truth, switch to 2D, show the metrics table — in under four minutes without notes. Do it three times.

Quick reference — the numbers to have memorised

Fact	Value
Modelled subdivisions	34 (not 36 — islands excluded)
Dataset	279,650 rows × 93 columns
Features	52, in 6 concept families
Splits	train 2016–20 / val 2021 / test 2022
Bust rate (test)	3.59%
Model AUROC (all leads)	0.840
Model AUROC (Day 3–7)	0.825
vs real IFS ENS	+0.025 raw / +0.040 calibrated
Decision cost reduction	–16.8%
ECE	0.011
OOD earned refusal	23.4% vs 3.4% ($\approx 7\times$)
Tests	68
Decisions logged	D-001 ... D-018

If you only have one evening

Do this, in order (~2 hours):

1. `TEAMMATE_BRIEFING.md` §0 and §1 (10 min)
2. Day 1's demo: `replay_demo.py` + click through the 3D dashboard (20 min)
3. Day 2's reading: `src/fbd/labels/bust.py` (30 min) — **the single most important file**
4. Day 3's poison test: run it, read it (20 min)
5. `TEAMMATE_BRIEFING.md` §15 — the 20 judge answers, out loud (40 min)

That gets you to "can defend it". The full five days get you to "can extend it".